ภาควิชาเทคโนโลยีสารสนเทศ

โครงงานปลายภาค

Full Stack Web Development

ออกแบบและพัฒนาเว็บแอปพลิเคชันที่ขับเคลื่อนด้วยฐานข้อมูล

`Express.js | EJS | Sequelize | SQLite | CSS`

ปีการศึกษา 2568 ภาคเรียนที่ 2

กำหนดส่ง: _____

โครงงานเดี่ยว • 100 คะแนน

1. ภาพรวมของโครงงาน

โครงงานปลายภาคนี้กำหนดให้นักศึกษาออกแบบและพัฒนาเว็บแอปพลิเคชันแบบ Full Stack โดยเลือกหัวข้อด้วยตนเอง ออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีอย่างน้อย 4 ตาราง พัฒนาระบบ CRUD ครบถ้วน และสร้างหน้ารายงาน (Report) 2 หน้าที่มีความหมาย โครงงานครอบคลุมตั้งแต่การออกแบบฐานข้อมูลไปจนถึงหน้าจอแสดงผลที่สวยงามและใช้งานได้จริง

> ⚠ **เลือกหัวข้อโครงงานด้วยตัวเอง - ห้ามใช้ระบบลงทะเบียนเรียน**
>
> **นักศึกษาแต่ละคนต้องเลือกหัวข้อโครงงานที่ไม่ซ้ำกันของตนเอง** โดย **ห้าม** ใช้ระบบลงทะเบียนเรียน (Student Enrollment) ที่ให้ไว้เป็นตัวอย่างในเอกสารนี้ เพราะเป็นเพียงตัวอย่างอ้างอิงเท่านั้น
>
> หัวข้อที่เลือกต้องได้รับการอนุมัติจากอาจารย์ก่อนเริ่มพัฒนา และห้ามใช้หัวข้อซ้ำกันระหว่างนักศึกษา
>
> ตลอดทั้งเอกสารนี้ ส่วนที่ระบุว่า ▯ ***ตัวอย่างเท่านั้น*** คือตัวอย่างระบบลงทะเบียนเรียนที่ใช้อ้างอิงเพื่อแสดงให้เห็นภาพ นักศึกษาต้องปรับใช้กับหัวข้อของตนเอง

2. การเลือกหัวข้อโครงงาน

แอปพลิเคชันที่เลือกต้องจัดการข้อมูลในโลกจริงที่มีตารางเชิงสัมพันธ์อย่างน้อย 4 ตาราง มีความสัมพันธ์แบบ One-to-Many อย่างน้อย 1 คู่ และแบบ Many-to-Many อย่างน้อย 1 คู่ (ผ่าน Junction Table) รองรับการทำ CRUD ครบถ้วน และมีรายงาน 2 หน้าที่มีประโยชน์

2.1 แนวคิดหัวข้อ (ตัวอย่าง)

ต่อไปนี้คือตัวอย่างหัวข้อเพื่อให้แนวคิด ไม่จำกัดเฉพาะรายการนี้ สามารถเสนอหัวข้ออื่นๆ ที่สนใจได้:

1. ระบบจัดการห้องสมุด - หนังสือ, ผู้แต่ง, สมาชิก, รายการยืม-คืน
2. ระบบสั่งอาหารร้านอาหาร - เมนู, หมวดหมู่, ลูกค้า, รายการสั่ง/รายละเอียดสั่ง
3. ระบบจองโรงแรม - ห้องพัก, ประเภทห้อง, แขก, การจอง
4. ระบบร้านค้าออนไลน์ - สินค้า, หมวดหมู่, ลูกค้า, คำสั่งซื้อ/รายการสั่ง

5. ระบบฟิตเนส/ยิม - สมาชิก, เทรนเนอร์, คลาส, การลงทะเบียนคลาส
6. ระบบจัดการอีเวนต์ - กิจกรรม, สถานที่, ผู้เข้าร่วม, การลงทะเบียน
7. คลินิกสัตว์เลี้ยง - สัตว์, เจ้าของ, สัตวแพทย์, นัดหมาย
8. ระบบเช่าหนัง/สตรีมมิ่ง - หนัง, ประเภท, ผู้ใช้, ประวัติการเช่า/รับชม
9. ระบบจัดการงานพนักงาน - พนักงาน, แผนก, โครงการ, การมอบหมายงาน
10. ระบบสูตรอาหาร/รีวิว - สูตร, วัตถุดิบ, ผู้ใช้, รายการโปรด/รีวิว

2.2 การขออนุมัติหัวข้อ

ส่งหัวข้อที่ต้องการเสนอให้อาจารย์อนุมัติก่อนเริ่มพัฒนา โดยข้อเสนอต้องประกอบด้วย:

- ชื่อแอปพลิเคชันและคำอธิบายสั้นๆ (1-2 ประโยค)
- รายชื่อตาราง 4 ตารางพร้อมคอลัมน์หลัก
- อธิบายความสัมพันธ์ระหว่างตาราง (Relationships)
- คำอธิบายสั้นๆ เกี่ยวกับรายงาน 2 หน้าที่วางแผนจะทำ

กำหนดส่งข้อเสนอ: _____ (ส่งก่อนเริ่มพัฒนา)

3. วัตถุประสงค์การเรียนรู้

เมื่อทำโครงงานนี้เสร็จสมบูรณ์แล้ว นักศึกษาจะสามารถ:

1. ออกแบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่มี Normalization ที่เหมาะสม
2. พัฒนา Backend ด้วย Node.js/Express.js ตามแพทเทิร์น MVC
3. ใช้ Sequelize ORM กำหนด Model, Association และทำงานกับฐานข้อมูล SQLite
4. สร้างหน้าเว็บแบบ Dynamic ด้วย EJS Template Engine
5. สร้างหน้า CRUD (Create, Read, Update, Delete) ครบถ้วนสำหรับทุกตาราง
6. พัฒนาหน้ารายงานที่รวบรวมและแสดงข้อมูลข้ามตารางด้วย Joins
7. ใช้ CSS จัดรูปแบบหน้าเว็บให้สวยงาม Responsive และใช้งานง่าย

4. เทคโนโลยีที่ใช้

ชั้น (Layer)	เทคโนโลยี	รายละเอียด
หน้าบ้าน (Frontend)	HTML5 + EJS	โครงสร้างหน้าเว็บและ Template Engine ฝั่งเซิร์ฟเวอร์
หน้าบ้าน (Frontend)	CSS3	จัดรูปแบบ, Layout (Flexbox/Grid), Responsive Design
หลังบ้าน (Backend)	Express.js	Node.js Web Framework สำหรับ Routing และ Middleware
ORM	Sequelize	Object-Relational Mapping สำหรับจัดการฐานข้อมูล
ฐานข้อมูล (Database)	SQLite	ฐานข้อมูลเชิงสัมพันธ์แบบไฟล์ เบาและพกพาสะดวก
Runtime	Node.js	JavaScript Runtime (แนะนำ v18 ขึ้นไป)

4.1 แพ็คเกจ npm ที่ต้องใช้

```
npm init -y
npm install express ejs sequelize sqlite3
npm install express-session connect-flash method-override
npm install dotenv nodemon --save-dev
```

5. ตัวอย่างอ้างอิง: ระบบลงทะเบียนเรียน

⚠ **ส่วนนี้เป็นตัวอย่างอ้างอิงเท่านั้น - ห้ามใช้ส่งเป็นโครงงาน**

ระบบลงทะเบียนเรียน (Student Enrollment) ด้านล่างนี้แสดงเพื่อเป็นแนวทางให้เห็นภาพของขอบเขต โครงสร้าง และคุณภาพที่คาดหวังเท่านั้น

ศึกษาตัวอย่างนี้ให้เข้าใจอย่างละเอียด
แล้วปรับใช้แพทเทิร์นเดียวกันกับหัวข้อของตัวเอง

5.1 ตัวอย่างการออกแบบฐานข้อมูล

*ℹ **ตัวอย่างเท่านั้น:** โครงสร้างฐานข้อมูล 4 ตารางด้านล่างนี้เป็นของระบบลงทะเบียน โครงงานของคุณต้องมีโครงสร้างใน ลักษณะเดียวกัน*

ระบบตัวอย่างประกอบด้วย 4 ตาราง โดยตาราง Enrollments เป็น Junction Table เชื่อมระหว่าง Students กับ Sections แบบ Many-to-Many:

ความสัมพันธ์: Students ←→ Enrollments ←→ Sections → Courses

One-to-Many: หนึ่งรายวิชา (Course) มีได้หลายกลุ่มเรียน (Section)

Many-to-Many: Students และ Sections เชื่อมกันผ่านตาราง Enrollments

ตารางตัวอย่างที่ 1: Students (นักศึกษา)

คอลัมน์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
id	INTEGER	PK, Auto Increment	รหัสนักศึกษา
studentCode	STRING(15)	NOT NULL, UNIQUE	รหัสนักศึกษา เช่น s6601012630001
firstName	STRING(100)	NOT NULL	ชื่อ
lastName	STRING(100)	NOT NULL	นามสกุล
email	STRING(150)	NOT NULL, UNIQUE	อีเมล
phone	STRING(20)	ALLOW NULL	เบอร์โทรศัพท์

ตารางตัวอย่างที่ 2: Courses (รายวิชา)

คอลัมน์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
id	INTEGER	PK, Auto Increment	รหัสรายวิชา
courseCode	STRING(10)	NOT NULL, UNIQUE	รหัสวิชา เช่น CS101
courseName	STRING(200)	NOT NULL	ชื่อรายวิชา
credits	INTEGER	NOT NULL, DEFAULT 3	หน่วยกิต
description	TEXT	ALLOW NULL	คำอธิบายรายวิชา

ตารางตัวอย่างที่ 3: Sections (กลุ่มเรียน)

คอลัมน์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
id	INTEGER	PK, Auto Increment	รหัสกลุ่มเรียน
sectionNumber	STRING(5)	NOT NULL	หมายเลขกลุ่ม เช่น A, B, 01
semester	STRING(20)	NOT NULL	ภาคเรียน เช่น 2/2568
instructor	STRING(150)	NOT NULL	ชื่ออาจารย์ผู้สอน
maxSeats	INTEGER	NOT NULL, DEFAULT 40	จำนวนที่นั่งสูงสุด
schedule	STRING(100)	ALLOW NULL	วัน/เวลา เช่น จ-พ 9:00-10:30
room	STRING(50)	ALLOW NULL	ห้องเรียน
courseId	INTEGER	FK → Courses.id	Foreign Key ไปยังตาราง Courses

ตารางตัวอย่างที่ 4: Enrollments (การลงทะเบียน)

คอลัมน์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
id	INTEGER	PK, Auto Increment	รหัสการลงทะเบียน
enrollDate	DATEONLY	NOT NULL, DEFAULT NOW	วันที่ลงทะเบียน
grade	STRING(5)	ALLOW NULL	เกรด เช่น A, B+, B
status	STRING(20)	DEFAULT 'enrolled'	สถานะ: enrolled / withdrawn / completed
studentId	INTEGER	FK → Students.id	Foreign Key ไปยัง Students
sectionId	INTEGER	FK → Sections.id	Foreign Key ไปยัง Sections

5.2 ตัวอย่าง Sequelize Associations

> ℹ **ตัวอย่างเท่านั้น:** ปรับแพทเทิร์น Association นี้ให้เข้ากับตารางและความสัมพันธ์ของโครงงานคุณเอง

```
// □□□□□□□□□: Associations □□□□□□□□□□□□□□□□□□□□

// Course □□□□□□ Section
Course.hasMany(Section, { foreignKey: 'courseId', onDelete: 'CASCADE' });
Section.belongsTo(Course, { foreignKey: 'courseId' });

// Student □□□□□□□□□□ Section □□□□ Enrollment (Many-to-Many)
Student.belongsToMany(Section, { through: Enrollment, foreignKey:
'studentId' });
Section.belongsToMany(Student, { through: Enrollment, foreignKey:
'sectionId' });

// Associations □□□□□□□□□□□□□□□□□□ Enrollment
Student.hasMany(Enrollment, { foreignKey: 'studentId' });
Enrollment.belongsTo(Student, { foreignKey: 'studentId' });
Section.hasMany(Enrollment, { foreignKey: 'sectionId' });
Enrollment.belongsTo(Section, { foreignKey: 'sectionId' });
```

5.3 ตัวอย่างหน้ารายงาน

> ℹ **ตัวอย่างเท่านั้น:** โครงงานของคุณต้องมีรายงาน 2 หน้าที่เกี่ยวข้องกับหัวข้อของคุณ โดยใช้แพทเทิร์นคล้ายกัน

ตัวอย่างรายงานที่ 1: นักศึกษาในกลุ่มเรียน

Route: GET /reports/students-in-class

วัตถุประสงค์: แสดงรายชื่อนักศึกษาทั้งหมดที่ลงทะเบียนในกลุ่มเรียนที่เลือก

- Dropdown เลือกกลุ่มเรียน (รหัสวิชา - กลุ่ม - ภาคเรียน)
- แสดงตาราง: รหัสนักศึกษา, ชื่อ-สกุล, อีเมล, วันที่ลงทะเบียน, เกรด, สถานะ
- ส่วนหัว: ชื่อวิชา, อาจารย์, เวลาเรียน, ห้อง, จำนวนนักศึกษา/ที่นั่งสูงสุด

ตัวอย่าง Query:

```
const section = await Section.findByPk(sectionId, {
  include: [
    { model: Course },
```

```
    { model: Enrollment, include: [{ model: Student }] }
  ]
});
```

ตัวอย่างรายงานที่ 2: รายงานการลงทะเบียนของนักศึกษา

Route: GET /reports/student-enroll

วัตถุประสงค์: แสดงรายวิชาและกลุ่มเรียนทั้งหมดที่นักศึกษาคนหนึ่งลงทะเบียนอยู่

- Dropdown เลือกนักศึกษา (รหัส - ชื่อ-สกุล)
- แสดงตาราง: รหัสวิชา, ชื่อวิชา, กลุ่ม, ภาคเรียน, อาจารย์, เกรด, สถานะ
- ส่วนหัว: ข้อมูลนักศึกษา, หน่วยกิตรวม
- จัดกลุ่มตามภาคเรียนเพื่อความเป็นระเบียบ

ตัวอย่าง Query:

```
const student = await Student.findByPk(studentId, {
  include: [{
    model: Enrollment,
    include: [{ model: Section, include: [{ model: Course }] }]
  }]
});
```

6. ข้อกำหนดของโครงงาน

ไม่ว่าจะเลือกหัวข้อใด โครงงานทุกชิ้นต้องปฏิบัติตามข้อกำหนดต่อไปนี้:

6.1 ข้อกำหนดฐานข้อมูล

- ตารางเชิงสัมพันธ์อย่างน้อย 4 ตารางที่มีข้อมูลที่สมเหตุสมผล
- มีความสัมพันธ์แบบ One-to-Many อย่างน้อย 1 คู่
- มีความสัมพันธ์แบบ Many-to-Many อย่างน้อย 1 คู่ (ผ่าน Junction Table)
- ใช้ Primary Key, Foreign Key และ Constraints (NOT NULL, UNIQUE, DEFAULT) อย่างถูกต้อง
- กำหนด Sequelize Models พร้อม Associations อย่างถูกต้อง
- มี Seed Script (database/seed.js) พร้อมข้อมูลตัวอย่างอย่างน้อย 10-20 รายการต่อตารางหลัก

6.2 ข้อกำหนด CRUD

ทำระบบ Create, Read, Update, Delete ครบถ้วนสำหรับทุกตาราง โดยแต่ละตารางต้องมีเส้นทาง (Route) ตามแพทเทิร์นนี้:

> *ℹ **ตัวอย่างเท่านั้น:** Route ด้านล่างใช้ /<entity> เป็นตัวอย่าง ให้เปลี่ยนเป็นชื่อ Entity ของคุณเอง*

การทำงาน	HTTP Method	Route Pattern	View / การทำงาน
แสดงทั้งหมด	GET	/<entity>	index.ejs - แสดงข้อมูลทั้งหมดในตาราง
แสดงรายละเอียด	GET	/<entity>/:id	show.ejs - แสดงรายละเอียดข้อมูลเดี่ยว
ฟอร์มเพิ่มใหม่	GET	/<entity>/new	create.ejs - ฟอร์มเปล่าสำหรับเพิ่มข้อมูล
บันทึก	POST	/<entity>	ตรวจสอบ, บันทึก DB, Redirect ไปหน้ารายการ
ฟอร์มแก้ไข	GET	/<entity>/:id/edit	edit.ejs - ฟอร์มที่เติมข้อมูลเดิมไว้แล้ว
อัปเดต	PUT	/<entity>/:id	ตรวจสอบ, อัปเดต, Redirect ไปหน้ารายละเอียด

การทำงาน	HTTP Method	Route Pattern	View / การทำงาน
ลบ	DELETE	/<entity>/:id	ยืนยัน, ลบข้อมูล, Redirect ไปหน้ารายการ

ข้อกำหนดแต่ละหน้า

หน้ารายการ (Index Page):

- แสดงข้อมูลทั้งหมดในตาราง HTML ที่จัดรูปแบบสวยงาม
- ปุ่ม ดู, แก้ไข, ลบ ในแต่ละแถว
- ปุ่ม "เพิ่มใหม่" สำหรับไปหน้าฟอร์มเพิ่มข้อมูล
- ตารางที่มี Foreign Key: แสดงข้อมูลที่อ่านได้ (ชื่อ) ไม่ใช่แค่ ID ตัวเลข

ฟอร์มเพิ่ม / แก้ไข (Create / Edit Forms):

- HTML Validation ที่เหมาะสม (required, type, maxlength)
- ใช้ Dropdown สำหรับช่อง Foreign Key โดยดึงข้อมูลจากฐานข้อมูล
- แสดง Flash Messages สำเร็จหรือผิดพลาด
- ปุ่มยกเลิก และฟอร์มแก้ไขต้องเติมข้อมูลเดิมไว้แล้ว

หน้ารายละเอียด (Show Page):

- แสดงข้อมูลครบถ้วนในรูปแบบที่อ่านง่าย
- ข้อมูลที่เกี่ยวข้องแสดงชื่อที่อ่านได้ (ไม่ใช่แค่ ID)
- ปุ่มแก้ไข, ลบ, และกลับไปรายการ
- การลบต้องมี confirm() ก่อนดำเนินการ

6.3 ข้อกำหนดหน้ารายงาน (Reports)

ทำหน้ารายงาน 2 หน้าที่เกี่ยวข้องกับหัวข้อที่เลือก โดยแต่ละรายงานต้อง:

- ดึงข้อมูลข้ามตารางโดยใช้ Sequelize Eager Loading (include)
- มี Dropdown หรือตัวกรองเพื่อเลือกดูข้อมูลบางส่วน
- แสดงผลลัพธ์ในตารางที่จัดรูปแบบสวยงาม
- มีส่วนสรุป (จำนวนรวม, ยอดรวม หรือสถิติอื่นๆ)

7. โครงสร้างโปรเจกต์ (MVC Pattern)

> ℹ **ตัวอย่างเท่านั้น:** ชื่อไดเรกทอรีและไฟล์เป็นตัวอย่าง ให้เปลี่ยนชื่อ Entity ตามหัวข้อของคุณ เช่น books/, orders/

```
your-project-name/
├── app.js                    # Express app entry point
├── .env                      # □□□□□□ (PORT, DB path)
├── package.json
├── database/
│   ├── database.sqlite       # □□□□□□□□□□□□□□ SQLite
│   └── seed.js               # Script □□□□□□□□□□□□□□□□□□□
├── models/
│   ├── index.js              # Sequelize instance & associations
│   ├── <Model1>.js
│   ├── <Model2>.js
│   ├── <Model3>.js
│   └── <Model4>.js
├── routes/
│   ├── <entity1>Routes.js
│   ├── <entity2>Routes.js
│   ├── <entity3>Routes.js
│   ├── <entity4>Routes.js
│   └── reportRoutes.js
├── controllers/
│   ├── <entity1>Controller.js
│   ...                       # Controller □□□□□□□□□ Entity
│   └── reportController.js
├── views/
│   ├── partials/
│   │   ├── header.ejs        # HTML head, navbar □□□□
│   │   └── footer.ejs        # Footer □□□□
│   ├── home.ejs              # □□□□□□□□ / Dashboard
│   ├── <entity1>/            # index, create, edit, show
│   ├── <entity2>/            # □□□□□□□□□□ CRUD □□□□□□□□□
│   ├── <entity3>/
│   ├── <entity4>/
│   └── reports/
│       ├── report1.ejs
│       └── report2.ejs
└── public/
    ├── css/
    │   └── style.css
    └── js/
        └── main.js               # Client-side scripts (□□□□□)
```

8. ข้อกำหนดด้าน UI และ CSS

แอปพลิเคชันต้องมีหน้าตาที่สวยงาม สอดคล้อง และใช้งานง่าย สามารถใช้ CSS เขียนเองหรือใช้ CSS Framework ขนาดเบาก็ได้ แต่ต้องแสดงให้เห็นความเข้าใจในการจัดรูปแบบ:

8.1 ส่วนประกอบ UI ที่ต้องมี

- Navigation Bar: แสดงทุกหน้าผ่าน header.ejs มีลิงก์ไปหน้าหลัก, หน้า Entity ทุกตัว, และเมนูรายงาน
- Consistent Layout: ใช้ Partials (header/footer) ร่วมกันทุกหน้า
- Responsive Design: แสดงผลได้ทั้งบน Desktop และ Tablet โดยใช้ Flexbox หรือ Grid
- Data Tables: ตารางสวยงามมีสีสลับแถว มี Hover Effect
- Forms: มี Label ชัดเจน, Input Type ที่เหมาะสม, Focus State ชัดเจน
- Flash Messages: แถบแจ้งเตือนสำเร็จ (เขียว) และผิดพลาด (แดง)
- Buttons: สีที่เหมาะสม (น้ำเงิน=หลัก, เขียว=เพิ่ม, แดง=ลบ, เทา=ยกเลิก)
- การลบ: ต้องมี JavaScript confirm() ก่อนดำเนินการ

8.2 ข้อกำหนดขั้นต่ำ CSS

- Stylesheet ภายนอก (public/css/style.css) เชื่อมทุกหน้า
- CSS Rules/Selectors อย่างน้อย 15 ตัว
- ใช้ CSS Variables สำหรับสี, ฟอนต์, ระยะห่างที่สอดคล้องกัน
- ใช้ Flexbox หรือ CSS Grid อย่างน้อย 1 จุด
- จัดรูปแบบตาราง, ฟอร์ม, ปุ่ม, และ Navigation อย่างสวยงาม
- Hover และ Focus State สำหรับส่วนที่โต้ตอบได้

9. ข้อมูลตัวอย่าง (Seed Data)

สร้างไฟล์ database/seed.js เพื่อเติมข้อมูลตัวอย่างที่สมจริงสำหรับทดสอบและสาธิต:

- ตารางหลัก: อย่างน้อย 10 รายการต่อตาราง
- ตาราง Junction / ธุรกรรม: อย่างน้อย 15-20 รายการ

- ข้อมูลต้องสมจริงและหลากหลายพอที่จะสาธิตทุกฟีเจอร์และรายงานได้

เพิ่มคำสั่ง Seed ใน package.json:

```
"scripts": {
  "start": "node app.js",
  "dev": "nodemon app.js",
  "seed": "node database/seed.js"
}
```

10. ฟีเจอร์เสริม (คะแนนพิเศษ, ไม่บังคับ)

นักศึกษาที่ต้องการทำเกินข้อกำหนดขั้นต่ำ
สามารถทำฟีเจอร์ต่อไปนี้เพื่อรับคะแนนพิเศษสูงสุด 10 คะแนน:

- ค้นหาและแบ่งหน้า (Search & Pagination) - 5 คะแนน
- Validation Middleware ด้วย express-validator - 3 คะแนน
- Dashboard หน้าแรกแสดงสถิติรวมและกิจกรรมล่าสุด - 3 คะแนน
- Dark Mode สลับโหมดมืด/สว่าง ด้วย CSS + localStorage - 2 คะแนน
- รายงานพร้อมพิมพ์ ด้วย @media print - 2 คะแนน

11. เกณฑ์การให้คะแนน

โครงงานจะถูกประเมินตามเกณฑ์ต่อไปนี้ รวม 100 คะแนน:

เกณฑ์การให้คะแนน	คะแนน	รายละเอียด
การออกแบบฐานข้อมูลและ Schema	15	ER Diagram, ตาราง 4 ตาราง, Normalization, Sequelize Models
Express.js Backend และ Routes	20	RESTful Routes, Controllers, Error Handling, Middleware
EJS Frontend และหน้า CRUD	20	หน้า CRUD ครบถ้วน, ฟอร์ม, EJS Template, Flash Messages
หน้ารายงาน (Reports)	15	รายงาน 2 หน้าทำงานถูกต้อง, กรองข้อมูล, Join ข้อมูล
CSS และการออกแบบ UI/UX	10	Responsive, สวยงาม, ใช้งานง่าย
คุณภาพโค้ดและโครงสร้างโปรเจกต์	10	MVC Pattern, โค้ดสะอาด, Comment, จัดไฟล์เป็นระบบ
เอกสารและการนำเสนอ	10	README, วิธีติดตั้ง, Screenshots, นำเสนอสด
รวม	100	

12. สิ่งที่ต้องส่ง

ส่งรายการต่อไปนี้ภายในกำหนดส่ง:

1. ซอร์สโค้ดทั้งหมด: ไฟล์โครงงานทั้งหมดในไฟล์ ZIP หรือลิงก์ Git Repository
2. README.md: วิธีติดตั้ง, คำอธิบายโครงงาน, ภาพหน้าจอหลัก, รายการฟีเจอร์ที่ทำ
3. ER Diagram: ไดอะแกรมความสัมพันธ์ของฐานข้อมูล (รูปภาพหรือ PDF) สร้างด้วย dbdiagram.io, draw.io
4. Seed Script: ไฟล์เติมข้อมูลตัวอย่างที่ทำงานได้
5. นำเสนอสด: เตรียมสาธิตแอปพลิเคชัน 10 นาทีในวันส่งงาน

13. ข้อปฏิบัติในการส่งงาน

1. โครงงานนี้เป็นงานเดี่ยว โค้ดทั้งหมดต้องเป็นผลงานของตัวเอง

2. **ห้าม** ใช้ระบบลงทะเบียนเรียน (Student Enrollment) ที่ให้ไว้ในเอกสารนี้เป็นโครงงาน
3. สามารถอ้างอิงบทเรียนออนไลน์และเอกสารได้ แต่ห้ามคัดลอกโค้ดจากแหล่งภายนอกหรือจากเพื่อนร่วมชั้น หากพบจะได้ 0 คะแนน
4. อนุญาตให้ใช้ AI (GitHub Copilot, ChatGPT ฯลฯ) เพื่อเรียนรู้และแก้บั๊ก แต่ต้องเข้าใจและอธิบายโค้ดทุกบรรทัดได้
5. ตั้งชื่อไฟล์ ZIP: รหัสนักศึกษา_ชื่อสกุล_TermProject.zip
6. ส่งช้าหักคะแนนวันละ 10 คะแนน สูงสุด 3 วัน หลังจากนั้นไม่รับ
7. ฟีเจอร์ทั้งหมดต้องทำงานได้จริงในวันสาธิต ฟีเจอร์ที่ทำงานไม่ได้จะไม่ได้รับคะแนน

ขอให้โชคดีและสนุกกับการเขียนโค้ด!

หากมีคำถาม สามารถสอบถามได้ในชั่วโมง Office Hours หรือโพสต์ในฟอรัมรายวิชา